ROLE 1: FRONT-END DEVELOPER

🎯 1. Purpose of the Role

The **Front-End Developer** is responsible for building the **entire user interface (UI)** of the VID system.

This includes:

- All **workspaces dashboards**
- Navigation (sidebar, routing)
- Forms, tables, reports
- User interactions across roles (Student, Faculty, Admin, Parent)

👉 In simple terms:
They build what users see, click, and interact with.

🧩 2. Core Responsibilities

🔹 UI Development

- Build **responsive dashboards** for all 26 workspaces
- Create reusable UI components:
  - Tables
  - Forms
  - Cards
  - Modals
  - Charts

🔹 Navigation System

- Sidebar (workspace-based navigation)
- Role-based menu visibility
- Breadcrumbs and page transitions

🔹 Forms & Data Input

- Admission forms
- Academic setup forms
- Fee entry forms
- Attendance marking UI
- Request & approval forms

🔹 Data Visualization

- Charts (attendance trends, fees, performance)
- Reports dashboards
- Analytics UI for Principal/Admin

🔹 **API Integration (with Backend)**

- Fetch data from APIs
- Display data dynamically
- Handle loading states, errors

🔹 **Role-Based UI Control (RBAC)**

- Show/hide UI based on roles:
  - Student
  - Faculty
  - Admin
  - Parent

🏫 **3. Workspace-wise Responsibilities**

🔴 **Academic Workspace**

- Academic dashboard UI
- Academic setup forms (classes, sections)
- Courses & master courses UI
- Assessment & question bank UI
- Calendar & timetable slots UI

🔵 **Student Workspace**

- Student dashboard
- Attendance view UI
- Report card UI
- Timetable display
- Fee status UI

🟢 **Faculty Workspace**

- Attendance marking UI
- Assignment creation UI
- Student performance entry UI
- Communication interface

🟡 **Examination Workspace**

- Exam schedule UI
- Marks entry interface
- Result display UI
- Report card generation view

🟠 **Admission Workspace**

- Application forms UI
- Admission pipeline UI
- Student onboarding interface

💰 **Payment Workspace**

- Fee dashboard UI
- Transaction history UI
- Reports & statements UI
- Scholarship/concession UI

👩‍💼 **HRMS & Employee**

- Employee dashboard UI
- Leave & attendance UI
- Payroll display UI

🚍 **Transport Workspace**

- Route management UI
- Bus tracking dashboard
- Student allocation UI

📚 **Library Workspace**

- Book inventory UI
- Issue/return UI

🏥 **Health Workspace**

- Medical records UI
- Alerts dashboard

🏠 **Hostel Workspace**

- Room allocation UI
- Attendance UI
- Gate pass interface

👪 Parent Portal

- Student progress UI
- Fee payment UI
- Communication UI

🧠 Team Owner Workspace

- Institution creation UI
- Subscription UI
- Global dashboard UI

🔐 Common Workspace (UMS)

- User management UI
- Requests & approvals UI
- Notice board UI

🤖 4. AI Integration Responsibilities

💬 Chatbot UI

- Chat interface (like WhatsApp)
- Message bubbles
- Input box + suggestions

🎤 Voice Agent UI

- Microphone button
- Voice input indicator
- Response display

📷 Face Attendance UI

- Camera interface
- Capture preview
- Status display (Present/Absent)

⚙️ 5. Tech Responsibilities

- Use frameworks like:
  - React / Next.js (recommended)
- State management (Redux / Context)
- API calls (Axios / Fetch)
- UI frameworks (Tailwind / Material UI)

🔄 6. Integration Flow

Frontend connects with:

👉 Backend APIs
👉 Authentication system
👉 Database (indirectly via APIs)
👉 AI services (chatbot, voice, face)

📌 7. Daily Work Style (In Your MVP Plan)

On a given day, FE will:

- Build UI for a workspace/module
- Integrate API
- Test UI flow
- Fix UI bugs

🚀 8. Output of Front-End Developer

At MVP completion:

- Fully functional UI for all workspaces
- Responsive dashboards
- Integrated forms & reports
- AI interaction interfaces

⚡ Simple Understanding

👉 Frontend = “Face of VID System”
👉 Without FE → System exists but users cannot use it

ROLE 2: BACK-END DEVELOPER

🎯 1. Purpose of the Role

The **Back-End Developer** is responsible for building the **core logic, APIs, and system behavior** of VID.

This includes:

- Business logic for all workspaces
- API development
- Authentication & authorization
- Data processing and validations

👉 In simple terms:
They build how the system works behind the scenes.

🧩 2. Core Responsibilities

🔹 API Development

- Build REST APIs for all modules:
  - Students
  - Faculty
  - Academics
  - Payments
  - Exams
  - etc.

🔹 Business Logic Implementation

- Academic rules (grading, attendance)
- Fee calculations (installments, fines)
- Admission workflows
- Report generation logic

🔹 Authentication & Security

- Login system (JWT/session-based)
- Role-Based Access Control (RBAC)
- Secure endpoints per role

🔹 Data Handling

- Process incoming data from frontend

- Validate inputs
- Send structured responses

🔹 **Integration Layer**

- Connect:
  - Frontend ↔ Backend
  - Backend ↔ Database
  - Backend ↔ AI services

🏫 **3. Workspace-wise Responsibilities**

🔴 **Academic Workspace**

- APIs for:
  - Academic setup
  - Courses & subjects
  - Sections & classes
  - Academic calendar
- Logic for:
  - Subject–faculty mapping
  - Curriculum structure

🔵 **Student Workspace**

- APIs for:
  - Student profiles
  - Attendance records
  - Report cards
- Logic for:
  - Student lifecycle
  - Data aggregation (dashboard)

🟢 **Faculty Workspace**

- APIs for:
  - Attendance marking
  - Assignment creation
  - Marks entry
- Logic for:
  - Class-wise data filtering
  - Performance updates

🟡 **Examination Workspace**

- APIs for:
  - Exam scheduling
  - Marks entry
  - Results publishing
- Logic for:
  - Grade calculation
  - Report card generation
  - Rank/toppers analysis

🟠 **Admission Workspace**

- APIs for:
  - Leads & enquiries
  - Applications
  - Admission approval
- Logic for:
  - Admission pipeline flow
  - Status tracking

💰 **Payment Workspace**

- APIs for:
  - Fee structure
  - Transactions
  - Reports
- Logic for:
  - Fee calculation
  - Discounts/concessions
  - Payment validation

👷 **HRMS Workspace**

- APIs for:
  - Employee records
  - Leave & attendance
  - Payroll
- Logic for:
  - Salary calculation
  - Leave approvals

🚍 **Transport Workspace**

- APIs for:

    - Routes & buses
    - Student allocation
- Logic for:
    - Route optimization (basic MVP)
    - Capacity handling

📚 **Library Workspace**

- APIs for:
    - Book inventory
    - Issue/return
- Logic for:
    - Due dates
    - Fine calculation

🏥 **Health Workspace**

- APIs for:
    - Medical records
    - Alerts
- Logic for:
    - Emergency logs

🏠 **Hostel Workspace**

- APIs for:
    - Room allocation
    - Gate pass
- Logic for:
    - Entry/exit tracking

👪 **Parent Portal**

- APIs for:
    - Student data view
    - Communication
- Logic for:
    - Parent-student linking

🧠 **Team Owner Workspace**

- APIs for:
    - Institution creation
    - Subscription plans

- Logic for:
  - Multi-tenant handling

🔐 **UMS (User Management System)**

- APIs for:
  - User creation
  - Role assignment
- Logic for:
  - Permissions mapping (RBAC)

🤖 **4. AI Integration Responsibilities**

💬 **Chatbot**

- API endpoint for chatbot queries
- Connect AI model with system data
- Provide responses based on:
  - Student data
  - Fees
  - Attendance

🎤 **Voice Agent**

- APIs to receive voice-translated text
- Trigger system actions (commands)

📷 **Face Attendance**

- API for:
  - Face match validation
  - Attendance marking
- Integration with AI model

⚙️ **5. Tech Responsibilities**

- Frameworks:
  - Node.js / Django / FastAPI
- API design:
  - REST / GraphQL
- Authentication:
  - JWT / OAuth
- Middleware handling

🔄 6. System Flow Responsibility

Backend manages:

User Request
→ Authentication
→ Role Validation
→ Business Logic
→ Database Query
→ Response

📌 7. Daily Work Style (MVP)

On a given day, BE will:

- Build APIs
- Implement logic
- Test endpoints
- Fix bugs

🚀 8. Output of Back-End Developer

At MVP completion:

- Fully functional APIs for all workspaces
- Secure authentication system
- Business logic implemented
- AI integrations enabled

⚡ Simple Understanding

👉 Backend = “Brain of VID System”
👉 Without BE → UI exists but nothing works

ROLE 3: FULL STACK DEVELOPER

🎯 1. Purpose of the Role

The **Full Stack Developer (FS)** acts as the **bridge between Front-End and Back-End**.

They ensure:

- UI connects correctly to APIs
- Data flows properly across the system
- Features work end-to-end

👉 In simple terms:
They make the entire system work together as one complete product.

🧩 2. Core Responsibilities

🔹 End-to-End Integration

- Connect Frontend UI with Backend APIs
- Ensure correct request/response handling
- Manage data flow across modules

🔹 Feature Completion

- Take partially built modules (FE + BE)
- Convert them into **fully working features**

🔹 Routing & Navigation Logic

- Handle:
  - Page routing
  - Role-based redirection
  - Workspace navigation

🔹 State & Data Management

- Maintain global state:
  - Logged-in user
  - Active workspace
  - Selected student/course/etc.

🔹 Debugging & Fixing

- Fix integration issues:
  - API mismatch
  - UI not rendering data
  - Broken workflows

🏫 3. Workspace-wise Responsibilities

🔴 Academic Workspace

- Connect:
  - Academic UI ↔ Academic APIs
- Ensure:
  - Courses display correctly
  - Setup forms save properly
  - Calendar sync works

🔵 Student Workspace

- Integrate:
  - Attendance API → UI display
  - Report card API → charts/tables
- Ensure smooth student dashboard experience

🟢 Faculty Workspace

- Connect:
  - Attendance marking → backend
  - Assignment → submission flow
- Ensure:
  - Real-time updates for faculty actions

🟡 Examination Workspace

- Integrate:
  - Marks entry → database
  - Results → student dashboard
- Ensure:
  - No data mismatch in reports

🟠 Admission Workspace

- Connect:
  - Application forms → APIs
- Ensure:

    - Status updates reflect instantly

💰 Payment Workspace

- Integrate:
    - Payment APIs → UI
- Ensure:
    - Transaction success/failure flows
    - Receipt generation

👩‍💼 HRMS Workspace

- Connect:
    - Leave requests → approvals
- Ensure:
    - Employee dashboards update correctly

🚌 Transport Workspace

- Integrate:
    - Route data → UI
- Ensure:
    - Student allocations reflect properly

📚 Library Workspace

- Connect:
    - Issue/return APIs → UI
- Ensure:
    - Book status updates correctly

🏠 Hostel Workspace

- Integrate:
    - Gate pass requests → approvals
- Ensure:
    - Logs are updated in real-time

👪 Parent Portal

- Connect:
    - Student data → parent view
- Ensure:
    - Correct data visibility

🧠 Team Owner Workspace

- Integrate:
  - Institution creation → backend
- Ensure:
  - Multi-tenant flow works correctly

🔐 UMS Workspace

- Connect:
  - User roles → permissions
- Ensure:
  - Correct access across all workspaces

🤖 4. AI Integration Responsibilities

💬 Chatbot Integration

- Connect chatbot UI ↔ backend ↔ AI engine
- Ensure:
  - Responses use real system data

🎤 Voice Agent Integration

- Convert voice input → backend commands
- Trigger actions like:
  - "Show attendance"
  - "Check fees"

📷 Face Attendance Integration

- Connect:
  - Camera UI → AI model → backend
- Ensure:
  - Attendance gets recorded properly

⚙️ 5. Tech Responsibilities

- Routing frameworks (React Router / Next routing)
- State management:
  - Redux / Context API
- API handling:
  - Axios / Fetch
- Integration testing tools

🔄 6. System Flow Ownership

Full Stack ensures this works perfectly:

Frontend UI
→ API Call
→ Backend Logic
→ Database
→ Response
→ UI Update

📌 7. Daily Work Style (MVP)

On a given day, FS will:

- Connect frontend with backend
- Fix integration issues
- Test complete workflows
- Ensure feature completeness

🚀 8. Output of Full Stack Developer

At MVP completion:

- Fully integrated system
- All modules working end-to-end
- Smooth user flows across workspaces
- AI features properly connected

⚡ Simple Understanding

👉 Full Stack = "System Integrator"
👉 Without FS → FE & BE exist, but system is broken

ROLE 4: AI ENGINEER

🎯 1. Purpose of the Role

The **AI Engineer** is responsible for building the **YANTRA Intelligence Layer**, which powers:

- 💬 Chatbot
- 🎤 Voice Agent
- 📷 Face Recognition Attendance

👉 In simple terms:
They make the system intelligent, automated, and interactive.

🧩 2. Core Responsibilities

🔹 AI Model Development

- Design and implement:
  - NLP models (for chatbot)
  - Speech recognition models (for voice agent)
  - Computer vision models (for face recognition)

🔹 AI Integration with System

- Connect AI models with:
  - Backend APIs
  - Database
  - Frontend UI

🔹 Automation Logic

- Enable:
  - Auto-responses
  - Smart suggestions
  - Intelligent workflows

🔹 Training & Optimization

- Train models using:
  - Student queries
  - System data
- Improve:
  - Accuracy

- Speed
- Relevance

🤖 3. Feature-wise Responsibilities

💬 A. CHATBOT (YANTRA CHAT AI)

🔹 What It Does

- Answers user queries like:
  - "Show my attendance"
  - "What is my fee status?"
  - "When is my exam?"

🔹 AI Engineer Responsibilities

- Build NLP pipeline:
  - Intent recognition
  - Entity extraction
- Create knowledge mapping:
  - Map queries → system APIs
- Train chatbot:
  - Student queries
  - Faculty queries
  - Admin queries

🔹 Integration Work

- Connect chatbot with:
  - Student Workspace
  - Payment Workspace
  - Academic Workspace

🔹 Example Flow

User: "Show my attendance"
→ AI detects intent
→ Calls backend API
→ Returns data
→ Displays response

🎤 B. VOICE AGENT (YANTRA VOICE)

🔹 What It Does

- Converts voice → action

Example:

- "Open timetable"
- "Check fees"

🔹 **AI Engineer Responsibilities**

- Speech-to-Text integration
- Command recognition system
- Intent mapping

🔹 **Integration Work**

- Connect voice commands to:
  - Backend APIs
  - UI navigation

🔹 **Example Flow**

User speaks
→ Voice converted to text
→ Intent detected
→ API triggered
→ Action performed

📷 C. FACE RECOGNITION ATTENDANCE

🔹 **What It Does**

- Automatically marks attendance using face detection

🔹 **AI Engineer Responsibilities**

- Build face recognition model:
  - Face detection
  - Face matching
- Create embeddings:
  - Store face data securely

🔹 **Integration Work**

- Connect:
  - Camera UI → AI model → Backend

🔹 **Example Flow**

Camera captures face
→ AI detects face
→ Matches with database
→ Marks attendance

🏫 **4. Workspace Integration Responsibilities**

AI Engineer integrates with all major workspaces:

🔴 **Student Workspace**

- Chatbot answers:
  - Attendance
  - Fees
  - Results

🟢 **Faculty Workspace**

- Voice commands:
  - "Mark attendance"
- Chatbot:
  - "Show class performance"

🟡 **Examination Workspace**

- Chatbot:
  - "Show results"
  - "Topper list"

💰 **Payment Workspace**

- Chatbot:
  - "Pending fees"
- Voice:
  - "Open payment page"

🧠 **Admin / Team Owner**

- AI insights:
  - Reports
  - Analytics queries

⚙️ **5. Tech Responsibilities**

- NLP frameworks:
  - OpenAI / Transformers
- Speech:
  - Whisper / Speech APIs
- Vision:
  - OpenCV / FaceNet / DeepFace

🔄 6. System Flow Ownership

AI handles:

User Input (Text/Voice/Image)
→ AI Processing
→ Intent Recognition
→ Backend API Call
→ Response

📌 7. Daily Work Style (MVP)

On a given day, AI Engineer will:

- Build AI features
- Train chatbot
- Integrate with backend
- Test AI responses

🚀 8. Output of AI Engineer

At MVP completion:

- Working chatbot across workspaces
- Voice command system
- Face attendance system
- AI integrated with full VID platform

⚡ Simple Understanding

👉 AI Engineer = "Intelligence of VID"
👉 Without AI → System works, but no automation or smart interaction

ROLE 5: DATABASE ENGINEER

🎯 1. Purpose of the Role

The **Database Engineer (DB Engineer)** is responsible for designing and managing the **entire data layer** of VID.

This includes:

- Database design (schemas, tables)
- Data relationships across workspaces
- Query optimization
- Data security & integrity

👉 In simple terms:
They build and manage where all system data is stored and how it is structured.

🧩 2. Core Responsibilities

🔹 Database Design

- Create structured schemas for:
  - Students
  - Faculty
  - Courses
  - Payments
  - Exams
  - etc.

🔹 Data Relationships

- Define relationships like:
  - Student ↔ Class
  - Faculty ↔ Subject
  - Student ↔ Payments
  - Student ↔ Attendance

🔹 Query Optimization

- Write efficient queries:
  - Fast data retrieval

    - Indexed queries
    - Aggregation queries

🔹 **Data Security**

- Implement:
    - Row-Level Security (RLS)
    - Tenant isolation
- Ensure:
    - One institution cannot access another's data

🔹 **Data Consistency**

- Maintain:
    - Data validation rules
    - Integrity constraints
    - Backup strategies

🏫 **3. Workspace-wise Responsibilities**

🔴 **Academic Workspace**

- Tables:
    - Classes
    - Sections
    - Subjects
    - Curriculum
- Relationships:
    - Class → Sections
    - Subject → Faculty

🔵 **Student Workspace**

- Tables:
    - Student profiles
    - Attendance records
    - Academic history
- Relationships:
    - Student → Class
    - Student → Attendance

🟢 **Faculty Workspace**

- Tables:

    - Faculty profiles
    - Assignments
    - Performance logs
- Relationships:
    - Faculty → Subjects
    - Faculty → Classes

🟡 **Examination Workspace**

- Tables:
    - Exams
    - Marks
    - Results
- Relationships:
    - Student → Marks
    - Exam → Subjects

🟠 **Admission Workspace**

- Tables:
    - Leads
    - Applications
    - Admission records
- Relationships:
    - Applicant → Admission status

💰 **Payment Workspace**

- Tables:
    - Fee structures
    - Transactions
    - Receipts
- Relationships:
    - Student → Payments

👩‍💼 **HRMS Workspace**

- Tables:
    - Employees
    - Attendance
    - Payroll
- Relationships:
    - Employee → Salary

🚌 Transport Workspace

- Tables:
  - Buses
  - Routes
  - Stops
- Relationships:
  - Student → Route

📚 Library Workspace

- Tables:
  - Books
  - Issue logs
- Relationships:
  - Student → Book issue

🏥 Health Workspace

- Tables:
  - Medical records
  - Alerts

🏠 Hostel Workspace

- Tables:
  - Rooms
  - Allocations
  - Gate pass logs

👪 Parent Portal

- Tables:
  - Parent profiles
  - Student linkage

🧠 Team Owner Workspace

- Tables:
  - Institutions
  - Subscriptions
- Relationships:
  - Institution → Users

🔐 UMS (User Management System)

- Tables:
  - Users
  - Roles
  - Permissions

🤖 4. AI Integration Responsibilities

💬 Chatbot

- Provide structured data access for:
  - Attendance
  - Fees
  - Results

🎤 Voice Agent

- Enable fast query retrieval for voice commands

📷 Face Attendance

- Store:
  - Face embeddings
  - Attendance logs

⚙️ 5. Tech Responsibilities

- Databases:
  - PostgreSQL / MySQL (recommended)
- Features:
  - Indexing
  - Views
  - Stored procedures
- Security:
  - RLS (Row-Level Security)

🔄 6. System Flow Ownership

Database handles:

Backend Request
→ Query Execution
→ Data Retrieval
→ Response

📌 7. Daily Work Style (MVP)

On a given day, DB Engineer will:

- Design tables
- Write queries
- Optimize performance
- Support backend APIs

🚀 8. Output of Database Engineer

At MVP completion:

- Fully structured database
- Optimized queries
- Secure multi-tenant data system
- Reliable data storage for all workspaces

⚡ Simple Understanding

👉 Database Engineer = “Data Foundation of VID”
👉 Without DB → No data, no system